Failure (accidentally blocking the event loop) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `blocking_event_loop_practice.py`: `time.sleep()` inside `async def` doesn't error, doesn't warn, and still gives a correct final answer — the only symptom is that everything took twice as long. **If not:** the first time you'd meet this bug would be in real code, as an unexplained slowdown with no obvious cause, instead of something you caused on purpose here and can now recognize immediately.

Basic Version

Approach 1 — the direct way

```
# blocking_event_loop_practice.py
import asyncio
import time

async def good_task():
    await asyncio.sleep(3)
    return "good"

async def blocking_task():
    time.sleep(3)
    return "blocking"

async def time_it(label, coro_a, coro_b):
    start = time.perf_counter()
    results = await asyncio.gather(coro_a, coro_b)
    elapsed = time.perf_counter() - start
    print(label, elapsed, results)

async def main():
    await time_it("two good tasks:", good_task(), good_task())
    await time_it("blocking + good:", blocking_task(), good_task())

asyncio.run(main())
```

Expected output (your exact numbers will vary slightly):

```
two good tasks: 3.01 ['good', 'good']
blocking + good: 6.02 ['blocking', 'good']
```

"Two good tasks" finishes in ~3 seconds — they genuinely overlapped. "Blocking + good" finishes in ~6 seconds — `time.sleep(3)` inside `blocking_task` froze the entire event loop for those 3 seconds, so `good_task` couldn't make any progress either, even though it was written correctly. No error, no warning — the only symptom is the doubled time.

Why the blocking task goes first: `gather` starts its tasks in the order you give them. If `good_task` went first, it would start its 3-second `asyncio.sleep` clock *before* `blocking_task` froze the loop — both waits would still overlap, and the total would be about 3 seconds, hiding the bug. With `blocking_task` first, the loop is frozen before `good_task` even starts, so the two waits add up to about 6 seconds.

This version works correctly and demonstrates the bug clearly. It's missing type hints and an `if __name__ == "__main__":` guard — both fine for a first working version.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — typed, with a `__main__` guard

```
# blocking_event_loop_practice.py
import asyncio
import time


async def good_task() -> str:
    await asyncio.sleep(3)
    return "good"


async def blocking_task() -> str:
    time.sleep(3)             # the bug: no await, freezes the whole loop
    return "blocking"


async def time_it(label: str, coro_a, coro_b) -> None:
    start = time.perf_counter()
    results = await asyncio.gather(coro_a, coro_b)
    elapsed = time.perf_counter() - start
    print(f"{label} took {elapsed:.2f}s -> {results}")


async def main() -> None:
    await time_it("two good tasks:", good_task(), good_task())
    await time_it("blocking + good:", blocking_task(), good_task())


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
two good tasks: took 3.01s -> ['good', 'good']
blocking + good: took 6.02s -> ['blocking', 'good']
```

Difference from Basic: full type hints document each function's contract. The `if __name__ == "__main__":` guard means this file can be imported (say, to reuse `time_it` in a later test) without the two 3-6 second demonstration runs firing automatically on import. Same two numbers either way — proving the exact same bug.

Approach 2 — `asyncio.to_thread`, the actual fix when you can't avoid a blocking call

Story: nobody writes `time.sleep(3)` inside `async def` on purpose in production — this bug happens by accident, almost always because a library only ships a synchronous version. **If not:** you'd have diagnosed the bug (Approach 1) with no way to actually fix it short of rewriting a library you don't own.

```
# blocking_event_loop_practice.py
import asyncio
import time


async def good_task() -> str:
    await asyncio.sleep(3)
    return "good"


async def blocking_task() -> str:
    time.sleep(3)
    return "blocking"


async def fixed_task() -> str:
    await asyncio.to_thread(time.sleep, 3)
    return "fixed"


async def time_it(label: str, coro_a, coro_b) -> None:
    start = time.perf_counter()
    results = await asyncio.gather(coro_a, coro_b)
    elapsed = time.perf_counter() - start
    print(f"{label} took {elapsed:.2f}s -> {results}")


async def main() -> None:
    await time_it("two good tasks:", good_task(), good_task())
    await time_it("blocking + good:", blocking_task(), good_task())
    await time_it("fixed + good (to_thread):", fixed_task(), good_task())
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
two good tasks: took 3.01s -> ['good', 'good']
blocking + good: took 6.02s -> ['blocking', 'good']
fixed + good (to_thread): took 3.02s -> ['fixed', 'good']
```

`fixed_task` still calls the exact same blocking `time.sleep(3)` internally — nothing about `time.sleep` itself changed. What changed is *where* it runs: `asyncio.to_thread(time.sleep, 3)` hands that call off to a separate worker thread and gives the event loop back something it can genuinely `await`, so `good_task` gets to run concurrently again, same as if `fixed_task` had used `asyncio.sleep` natively.

Approach 3 — the realistic version: a sync-only library call, not `time.sleep`

Story: plenty of real code accidentally calls a sync-only library (a database driver, a file-parsing library, `requests`) from inside `async def` — it blocks the event loop exactly as badly as `time.sleep`, just without a name that gives it away. **If not:** the first sync-only dependency you reach for in a real agent would silently freeze every other coroutine it's supposed to run alongside.

```
# blocking_event_loop_practice.py
import asyncio
import time
import requests   # a real, sync-only library -- no async version exists


def fetch_sync(url: str) -> int:
    """A normal, blocking function from a library never written to be async."""
    response = requests.get(url, timeout=10)
    return response.status_code


async def good_task() -> str:
    await asyncio.sleep(1)
    return "good"


async def fetch_status_blocking(url: str) -> int:
    # the bug, in its realistic form: calling a sync-only library directly
    # from inside async def, with no await protecting the event loop
    return fetch_sync(url)
```

```
async def fetch_status_fixed(url: str) -> int:
    # the fix: the exact same sync function, run off-thread
    return await asyncio.to_thread(fetch_sync, url)


async def main() -> None:
    start = time.perf_counter()
    results = await asyncio.gather(
        good_task(), fetch_status_fixed("https://example.com"),
    )
    print(
        f"good + fixed fetch: {time.perf_counter() - start:.2f}s -> {results}"
    )


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (needs internet and `pip install requests`; `gather` waits for the slower task, so the total is about 1 second — `good_task`'s wait overlaps with the request instead of adding to it):

```
good + fixed fetch: 1.00s -> ['good', 200]
```

This is the realistic version of the bug: nobody writes `time.sleep(3)` inside `async def` in production, but plenty of code accidentally calls a sync-only library (`requests`, a database driver without an async variant, a file-parsing library) the same way `fetch_status_blocking` does here — and it blocks the event loop exactly as badly, just without a name that gives it away. `fetch_status_fixed` fixes it the same way Approach 2 did: `asyncio.to_thread(...)` around the call, nothing else about the library or the call itself changes.

Difference from Approach 1, and between Approaches 2/3: Approach 1 only diagnoses and measures the bug. Approach 2 fixes the exact same `time.sleep(3)` call from the exercise, to prove the mechanism works. Approach 3 shows the fix applied to what this bug actually looks like in real code — a synchronous library function (`requests.get`, standing in for any sync-only dependency) called from inside `async def` without protection — which is the version of this mistake you'll actually make one day, not a `time.sleep()` typo.

Which one should you actually write? In real code: never call a known-blocking function directly from inside `async def` — either use that library's async equivalent if one exists (`httpx` / `aiohttp` instead of `requests`, as Doc09's Core Concepts note), or wrap the sync call in `asyncio.to_thread(...)` the way Approach 3 does when no async version exists. Reach for `asyncio.to_thread` specifically when you're stuck with a sync-only dependency you can't swap

out — it's the standard, minimal-effort fix, and exactly what protects an agent's event loop the moment one of its tools turns out to be a synchronous library call in disguise.